Ejercicio 1: Ficha de jugador (estructura de una clase y constructor)

Simulado: el jugador es un objeto en memoria. No hay base de datos ni archivos.

Enunciado: crea la clase `Jugador` con los atributos `nombre` (`String`), `dorsal` (`int`) y `goles` (`int`). Su constructor recibe el nombre y el dorsal e inicializa `goles` a 0. Añade el método `marcarGol()`, que suma 1 a `goles`, y el método `descripcion()`, que devuelve `#DORSAL NOMBRE - goles: N`. En `main`, crea a `"Paco Chilindrón"` con el dorsal 9 y a `"Manolo Manoplas"` con el dorsal 1. Haz que Paco marque 2 goles e imprime la descripción de ambos.

java

```
// En un proyecto real esta clase iría en el paquete futbol.modelo
class Jugador { // declaramos la clase Jugador (sin public: va en el mismo archivo)
    String nombre; // atributo: nombre del jugador
    int dorsal; // atributo: número de camiseta
    int goles; // atributo: goles marcados

    Jugador(String nombre, int dorsal) { // constructor: mismo nombre que la clase y sin tipo de retorno
        this.nombre = nombre; // this.nombre es el atributo y nombre es el parámetro
        this.dorsal = dorsal; // guardamos el dorsal recibido
        this.goles = 0; // todo jugador empieza con 0 goles
    } // cierra el constructor

    void marcarGol() { // método que modifica el estado del objeto
        goles++; // sumamos un gol
    } // cierra marcarGol

    String descripcion() { // método que devuelve un texto con los datos
        return "#" + dorsal + " " + nombre + " - goles: " + goles; // construimos la ficha
    } // cierra descripcion
} // cierra la clase Jugador

public class Main { // clase principal
    public static void main(String[] args) { // punto de entrada
        Jugador j1 = new Jugador("Paco Chilindrón", 9); // new crea el objeto y llama al constructor
```

```
        Jugador j2 = new Jugador("Manolo Manoplas", 1); // segundo objeto,
independiente del primero
        j1.marcarGol(); // primer gol de Paco
        j1.marcarGol(); // segundo gol de Paco
        System.out.println(j1.descripcion()); // ficha de Paco
        System.out.println(j2.descripcion()); // ficha de Manolo (caso límite: 0
goles)
    } // cierra main
} // cierra la clase
```

text

```
#9 Paco Chilindrón - goles: 2
#1 Manolo Manoplas - goles: 0
```

Explicación: una clase define atributos (datos) y métodos (comportamiento). Cada objeto creado con `new` tiene su propia copia de los atributos, por eso Manolo sigue en 0 goles aunque Paco marque. El constructor se llama como la clase, no declara tipo de retorno y se ejecuta una vez al crear el objeto. `this` distingue el atributo del parámetro cuando se llaman igual.

Ejercicio 2: Puntos del equipo (visibilidad y encapsulación)

Simulado: la clasificación es un objeto en memoria. No hay base de datos.

Enunciado: crea la clase `Equipo` con los atributos **privados** `nombre` y `puntos` (este empieza en 0). Su constructor recibe el nombre. Crea los métodos públicos `getNombre()` y `getPuntos()`. Crea `boolean registrarResultado(int puntos)`: solo acepta 0, 1 o 3. Si el valor es otro, devuelve `false` sin cambiar nada. Si es válido, lo suma y devuelve `true`. En `main`, crea `"Real Cebolleta"` y recorre con un `for` el array `{3, 1, 2, 0, 3}`.

- Si el resultado es válido, imprime `Jornada N: +P puntos, total T`.
- Si no lo es, imprime `Jornada N: resultado imposible (P puntos), se ignora`.

Al final imprime `Clasificación: NOMBRE con T puntos`.

java

```
// En un proyecto real esta clase iría en el paquete futbol.modelo
class Equipo { // declaramos la clase Equipo
    private String nombre; // private: solo accesible desde dentro de esta clase
    private int puntos; // private: nadie de fuera puede cambiarlo directamente

    Equipo(String nombre) { // constructor con el nombre del equipo
```

```
        this.nombre = nombre; // guardamos el nombre
        this.puntos = 0; // empieza sin puntos
    } // cierra el constructor

    public String getNombre() { // getter público del nombre
        return nombre; // devolvemos el valor privado
    } // cierra getNombre

    public int getPuntos() { // getter público de los puntos
        return puntos; // devolvemos el valor privado
    } // cierra getPuntos

    public boolean registrarResultado(int puntos) { // única vía para modificar los
puntos
        if (puntos != 0 && puntos != 1 && puntos != 3) { // en fútbol solo se ganan
0, 1 o 3 puntos
            return false; // rechazamos el valor y no tocamos el estado
        } // cierra el if
        this.puntos += puntos; // sumamos al total del equipo
        return true; // indicamos que se aceptó
    } // cierra registrarResultado
} // cierra la clase Equipo

public class Main { // clase principal
    public static void main(String[] args) { // punto de entrada
        Equipo equipo = new Equipo("Real Cebolleta"); // creamos el equipo
        int[] resultados = {3, 1, 2, 0, 3}; // puntos de cada jornada (el 2 es
imposible)
        for (int i = 0; i < resultados.length; i++) { // recorremos las jornadas
            boolean aceptado = equipo.registrarResultado(resultados[i]); //
intentamos registrar
            if (aceptado) { // si el método devolvió true
                System.out.println("Jornada " + (i + 1) + ": +" + resultados[i] + "
puntos, total " + equipo.getPuntos()); // éxito
            } else { // si devolvió false
                System.out.println("Jornada " + (i + 1) + ": resultado imposible ("
+ resultados[i] + " puntos), se ignora"); // rechazo
            } // cierra el if-else
        } // cierra el for
        System.out.println("Clasificación: " + equipo.getNombre() + " con " +
equipo.getPuntos() + " puntos"); // resumen final
        // equipo.puntos = 100; // esta línea NO compilaría: puntos es private y Main
es otra clase
```

```
    } // cierra main
} // cierra la clase
```

text

```
Jornada 1: +3 puntos, total 3
Jornada 2: +1 puntos, total 4
Jornada 3: resultado imposible (2 puntos), se ignora
Jornada 4: +0 puntos, total 4
Jornada 5: +3 puntos, total 7
Clasificación: Real Cebolleta con 7 puntos
```

Explicación: `private` impide que otras clases lean o modifiquen un atributo directamente. `public` lo permite. La encapsulación consiste en ocultar los datos y ofrecer métodos que garanticen reglas, como que solo se sumen 0, 1 o 3 puntos. Los getters permiten leer sin permitir escribir. El parámetro `puntos` oculta al atributo del mismo nombre, por eso se usa `this.puntos` para el atributo.

Ejercicio 3: Estadísticas de la jornada (miembros y métodos estáticos)

Simulado: el contador de la jornada es un campo `static` en memoria. No hay marcador real.

Enunciado: crea la clase `Partido` con los atributos `local`, `visitante`, `golesLocal` y `golesVisitante`. Añade dos atributos **estáticos**, `golesTotales` y `partidosJugados`, ambos a 0. El constructor recibe los cuatro datos y actualiza los dos contadores estáticos. El método `resultado()` devuelve `LOCAL GL-GV VISITANTE`. Crea el método estático `resumen()`, que devuelve `Partidos: P, goles: G, media entera: M`. La media es `golesTotales / partidosJugados` con división entera, o 0 si no hay partidos. En `main`, imprime primero `Antes de jugar ->` más `Partido.resumen()`. Crea después estos tres partidos: `"Cebolleta CF"` 2-1 `"Atlético Chorizo"`, `"Sporting Croqueta"` 3-3 `"Betis Tortilla"` y `"Racing Bocata"` 0-0 `"Celta Churro"`. Imprime el resultado de cada uno y, al final, `Tras la jornada ->` más `Partido.resumen()`.

java

```
// En un proyecto real esta clase iría en el paquete futbol.modelo
class Partido { // declaramos la clase Partido
    static int golesTotales = 0; // static: una única copia compartida por todos los partidos
    static int partidosJugados = 0; // static: contador compartido de partidos

    String local; // atributo de instancia: equipo local
    String visitante; // atributo de instancia: equipo visitante
```

```
    int golesLocal; // atributo de instancia: goles del local
    int golesVisitante; // atributo de instancia: goles del visitante

    Partido(String local, String visitante, int golesLocal, int golesVisitante) {
// constructor con los 4 datos
        this.local = local; // guardamos el local
        this.visitante = visitante; // guardamos el visitante
        this.golesLocal = golesLocal; // guardamos sus goles
        this.golesVisitante = golesVisitante; // guardamos los goles del visitante
        golesTotales += golesLocal + golesVisitante; // actualizamos el contador
compartido de goles
        partidosJugados++; // actualizamos el contador compartido de partidos
    } // cierra el constructor

    String resultado() { // método de instancia: usa los datos de este partido
        return local + " " + golesLocal + "-" + golesVisitante + " " + visitante;
// texto del marcador
    } // cierra resultado

    static String resumen() { // método estático: no necesita ningún objeto
        int media = 0; // media entera, 0 si no hay partidos
        if (partidosJugados > 0) { // evitamos dividir entre cero
            media = golesTotales / partidosJugados; // división entera
        } // cierra el if
        return "Partidos: " + partidosJugados + ", goles: " + golesTotales + ",
media entera: " + media; // texto del resumen
    } // cierra resumen
} // cierra la clase Partido

public class Main { // clase principal
    public static void main(String[] args) { // punto de entrada
        System.out.println("Antes de jugar -> " + Partido.resumen()); // se llama
con el nombre de la clase, sin objeto
        Partido p1 = new Partido("Cebolleta CF", "Atlético Chorizo", 2, 1); //
primer partido
        Partido p2 = new Partido("Sporting Croqueta", "Betis Tortilla", 3, 3); //
segundo partido
        Partido p3 = new Partido("Racing Bocata", "Celta Churro", 0, 0); // tercer
partido
        System.out.println(p1.resultado()); // marcador del primero
        System.out.println(p2.resultado()); // marcador del segundo
        System.out.println(p3.resultado()); // marcador del tercero
```

```
        System.out.println("Tras la jornada -> " + Partido.resumen()); // resumen
compartido
    } // cierra main
} // cierra la clase
```

text

```
Antes de jugar -> Partidos: 0, goles: 0, media entera: 0
Cebolleta CF 2-1 Atlético Chorizo
Sporting Croqueta 3-3 Betis Tortilla
Racing Bocata 0-0 Celta Churro
Tras la jornada -> Partidos: 3, goles: 9, media entera: 3
```

Explicación: un miembro `static` pertenece a la clase y no a cada objeto, así que todos los partidos comparten los mismos contadores. Un método estático se llama con `NombreDeClase.metodo()` y no puede usar atributos de instancia como `local`, porque no hay ningún objeto concreto. Los goles son 3 + 6 + 0 = 9 y 9 / 3 da 3. Antes de crear ningún partido el resumen funciona gracias a la comprobación contra la división por cero.

Ejercicio 4: Plantilla de fichajes (objetos de varias clases y paquetes)

Simulado: los paquetes (`futbol.modelo`, `futbol.club`, `futbol.app`) se indican solo con comentarios, porque el entorno exige un único archivo. Un `package` real no se puede usar aquí.

Enunciado: crea la clase `Fichaje` con `nombre` y `precio` (millones, `int`) privados, constructor y getters. Crea la clase `Club` con un array privado `Fichaje[] plantilla` de capacidad 3 y un contador `numero`. Añade `boolean fichar(Fichaje f)`: si hay sitio, guarda el fichaje y devuelve `true`. Si la plantilla está llena, devuelve `false`. Añade `int gastoTotal()`, que suma los precios de los fichajes guardados, y `getNumero()`. En `main`, crea el club y recorre estos cuatro candidatos: `Pepe Regate` (12), `Luis Cabezazo` (8), `Ñito Zurdo` (20) y `Tito Banquillo` (5).

- Si ficha, imprime `Fichado: NOMBRE (P M)`.
- Si no, imprime `Plantilla llena: no cabe NOMBRE`.

Al final imprime `Gasto total: G M en N fichajes`.

java

```
// En un proyecto real: package futbol.modelo; (clase Fichaje)
class Fichaje { // un fichaje es un objeto con nombre y precio
    private String nombre; // nombre privado
    private int precio; // precio privado en millones
```

```
    Fichaje(String nombre, int precio) { // constructor
        this.nombre = nombre; // guardamos el nombre
        this.precio = precio; // guardamos el precio
    } // cierra el constructor

    public String getNombre() { // getter del nombre
        return nombre; // devolvemos el nombre
    } // cierra getNombre

    public int getPrecio() { // getter del precio
        return precio; // devolvemos el precio
    } // cierra getPrecio
} // cierra la clase Fichaje

// En un proyecto real: package futbol.club; (clase Club, con import
futbol.modelo.Fichaje;)
class Club { // un club guarda un array de objetos Fichaje
    private Fichaje[] plantilla = new Fichaje[3]; // array de 3 huecos, todos null
al principio
    private int numero = 0; // cuántos huecos están ocupados

    public boolean fichar(Fichaje f) { // intenta añadir un fichaje
        if (numero == plantilla.length) { // caso límite: plantilla llena
            return false; // no cabe
        } // cierra el if
        plantilla[numero] = f; // guardamos el objeto en el primer hueco libre
        numero++; // un hueco más ocupado
        return true; // fichaje aceptado
    } // cierra fichar

    public int gastoTotal() { // suma los precios de los fichajes guardados
        int total = 0; // acumulador
        for (int i = 0; i < numero; i++) { // solo recorremos los huecos ocupados
            total += plantilla[i].getPrecio(); // sumamos el precio de cada fichaje
        } // cierra el for
        return total; // devolvemos el gasto
    } // cierra gastoTotal

    public int getNumero() { // getter del número de fichajes
        return numero; // devolvemos el contador
    } // cierra getNumero
} // cierra la clase Club
```

```
// En un proyecto real: package futbol.app; (clase Main, con imports de modelo y club)
public class Main { // clase principal
    public static void main(String[] args) { // punto de entrada
        Club club = new Club(); // creamos el club
        Fichaje[] candidatos = { // array de objetos Fichaje
            new Fichaje("Pepe Regate", 12), // primer candidato
            new Fichaje("Luis Cabezazo", 8), // segundo candidato
            new Fichaje("Ñito Zurdo", 20), // tercer candidato
            new Fichaje("Tito Banquillo", 5) // cuarto candidato (no cabrá)
        }; // cierra el array
        for (int i = 0; i < candidatos.length; i++) { // recorremos los candidatos
            Fichaje c = candidatos[i]; // candidato actual
            if (club.fichar(c)) { // si el club lo acepta
                System.out.println("Fichado: " + c.getNombre() + " (" + c.getPrecio() + " M)"); // éxito
            } else { // si la plantilla está llena
                System.out.println("Plantilla llena: no cabe " + c.getNombre()); // rechazo
            } // cierra el if-else
        } // cierra el for
        System.out.println("Gasto total: " + club.gastoTotal() + " M en " + club.getNumero() + " fichajes"); // resumen
    } // cierra main
} // cierra la clase
```

text

```
Fichado: Pepe Regate (12 M)
Fichado: Luis Cabezazo (8 M)
Fichado: Ñito Zurdo (20 M)
Plantilla llena: no cabe Tito Banquillo
Gasto total: 40 M en 3 fichajes
```

Explicación: un array de objetos solo guarda referencias, y empieza con todos los huecos en `null`. Por eso `gastoTotal` recorre hasta `numero` y no hasta `plantilla.length`, ya que `plantilla[2].getPrecio()` con un hueco vacío lanzaría `NullPointerException`. Aquí 12 + 8 + 20 = 40. Los paquetes (`package` e `import`) organizan las clases en carpetas lógicas y evitan choques de nombres.

Cómo sería con paquetes y Maven (teoría, no ejecutable aquí)

Este bloque es solo explicativo, porque un compilador online de un solo archivo no puede ejecutarlo. En un proyecto Maven real, cada clase iría en su propio archivo. La estructura sería esta:

text

```
futbol/
├── pom.xml
└── src/main/java/futbol/
    ├── modelo/Fichaje.java    (primera línea: package futbol.modelo;)
    ├── club/Club.java         (package futbol.club; import futbol.modelo.Fichaje;)
    └── app/Main.java          (package futbol.app; imports de las otras dos)
```

Un `pom.xml` mínimo de ejemplo sería este:

xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0">
  <modelVersion>4.0.0</modelVersion>
  <groupId>es.ejemplo</groupId>
  <artifactId>futbol</artifactId>
  <version>1.0</version>
  <packaging>jar</packaging>
  <properties>
    <maven.compiler.source>17</maven.compiler.source>
    <maven.compiler.target>17</maven.compiler.target>
  </properties>
</project>
```

Con Maven instalado, `mvn package` compila el proyecto y genera `target/futbol-1.0.jar`. Después, `java -cp target/futbol-1.0.jar futbol.app.Main` ejecuta el programa. Las clases usadas desde otro paquete deben ser `public`, y sus métodos también si se llaman desde fuera. Por eso en proyecto real habría que poner `public class Fichaje`, con cada clase en su archivo.

Ejercicio 5: El portero (herencia básica)

Simulado: los jugadores son objetos en memoria. Solo se adelanta lo mínimo de herencia para RA4 g): `extends`, `super(...)` y `@Override`. No hay clases abstractas, interfaces ni polimorfismo avanzado, que quedan para la UT10 (RA7).

Enunciado: crea la clase `Jugador` con `nombre` y `dorsal` privados, un constructor que recibe ambos y el método `descripcion()`, que devuelve `#DORSAL NOMBRE`. Crea la clase `Portero`, que **hereda** de `Jugador`. Añade el atributo `paradas` (empieza en 0), el método `parar()` (suma 1) y un constructor que llama a `super(nombre, dorsal)`. Sobrescribe `descripcion()` con `@Override`: debe devolver lo de `Jugador` más `- paradas: N`. En `main`, crea a `Paco Chilindrón` (9) como `Jugador`, a `Manolo Manoplas` (1) como `Portero` con 3 paradas y a `Suplente Sueñoso` (13) como `Portero` sin paradas. Imprime la descripción de los tres.

java

```
// En un proyecto real esta clase iría en el paquete futbol.modelo
class Jugador { // clase padre (superclase)
    private String nombre; // atributo privado del padre
    private int dorsal; // atributo privado del padre

    Jugador(String nombre, int dorsal) { // constructor del padre
        this.nombre = nombre; // guardamos el nombre
        this.dorsal = dorsal; // guardamos el dorsal
    } // cierra el constructor

    String descripcion() { // método que el hijo va a sobrescribir
        return "#" + dorsal + " " + nombre; // texto base del jugador
    } // cierra descripcion
} // cierra la clase Jugador

class Portero extends Jugador { // extends: Portero hereda de Jugador (subclase)
    private int paradas; // atributo propio del portero

    Portero(String nombre, int dorsal) { // constructor del hijo
        super(nombre, dorsal); // super llama al constructor del padre, y debe ser la primera línea
        this.paradas = 0; // inicializamos el atributo nuevo
    } // cierra el constructor

    void parar() { // método nuevo, solo existe en Portero
        paradas++; // sumamos una parada
    } // cierra parar

    @Override // indica al compilador que sobrescribimos un método del padre
    String descripcion() { // misma firma que en Jugador
        return super.descripcion() + " - paradas: " + paradas; // reutilizamos el texto del padre y añadimos las paradas
    } // cierra descripcion
```

```
} // cierra la clase Portero

public class Main { // clase principal
    public static void main(String[] args) { // punto de entrada
        Jugador j = new Jugador("Paco Chilindrón", 9); // jugador normal
        Portero p1 = new Portero("Manolo Manoplas", 1); // portero titular
        Portero p2 = new Portero("Suplente Sueñoso", 13); // portero que no para
nada
        p1.parar(); // primera parada
        p1.parar(); // segunda parada
        p1.parar(); // tercera parada
        System.out.println(j.descripcion()); // usa la versión del padre
        System.out.println(p1.descripcion()); // usa la versión sobrescrita del hijo
        System.out.println(p2.descripcion()); // caso límite: 0 paradas
    } // cierra main
} // cierra la clase
```

text

```
#9 Paco Chilindrón
#1 Manolo Manoplas - paradas: 3
#13 Suplente Sueñoso - paradas: 0
```

Explicación: `extends` hace que `Portero` herede los atributos y métodos de `Jugador`, y añada los suyos. Los atributos privados del padre no son accesibles desde el hijo, así que `Portero` usa `super(...)` para inicializarlos. `@Override` sobrescribe un método con la misma firma, y el compilador avisa si no coincide con ninguno del padre. `super.descripcion()` reutiliza el código del padre en lugar de repetirlo.